

Agent-in-the-Loop Forecasting

B Harish
harish1@iisc.ac.in

Abstract

The analyst-in-the-loop workflow made popular by Facebook’s Prophet forecasting system addresses a practical problem in large-scale time-series forecasting: fully automatic models are fast, but they often fail on data quality issues, changepoints, outliers, holiday effects, and domain-specific shocks. Human analysts can diagnose these issues, but expert time-series analysts do not scale. This proposal modernizes the original workflow by replacing the expert analyst with a bounded large-language-model (LLM) agent that acts as a junior forecasting analyst. The agent does not directly forecast the time series. Instead, it uses tools for drift detection, changepoint detection, outlier detection, model comparison, intervention recommendation, backtesting, and report generation. The core question is whether an LLM-based agent can improve the reliability, explainability, and maintainability of a forecasting pipeline by automating the diagnostic role traditionally played by a human analyst. The proposal does not claim that AI forecast agents are a completely new idea; rather, it studies a focused, small-scale version of an idea already appearing in industry and open-source systems.

1 Introduction to the Analyst-in-the-Loop Workflow

1.1 Motivation

Time-series forecasting is central to capacity planning, demand estimation, anomaly detection, budgeting, and operational decision-making. In many practical settings, the main difficulty is not fitting one model to one clean series but producing reliable forecasts across many heterogeneous, noisy, and evolving time series. Business series often contain multiple seasonalities, structural breaks, missing dates, outliers, irregular holidays, product launches, marketing campaigns, and changes in data collection.

Classical automatic forecasting methods are useful, but they may fail silently when the series has an unmodeled regime change or an unusual event. Human analysts can often recognize such issues from plots and business context, but skilled forecasting analysts are expensive and scarce. This creates the central tension: automation is necessary for scale, while human judgment is necessary for robustness.

1.2 The Prophet View of Analyst-in-the-Loop Forecasting

Taylor and Letham’s Prophet paper, *Forecasting at Scale*, is a useful reference point for this project [1]. The paper not only introduces a model, but also introduces a workflow for practical forecasting at organizational scale. The workflow can be summarized as follows:

1. Fit an interpretable, configurable forecasting model.
2. Generate forecasts and compare them against reasonable baselines.
3. Use simulated historical forecasts to estimate out-of-sample performance.
4. Automatically flag suspicious forecasts or large errors.

5. Ask an analyst to inspect the flagged cases, diagnose the likely failure mode, and adjust the model or data treatment.
6. Refit and re-evaluate the forecast after the intervention.

The Prophet model itself is decomposable:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t, \quad (1)$$

where $g(t)$ is the trend, $s(t)$ captures seasonality, $h(t)$ captures holiday effects, and ϵ_t is the residual error. This decomposition is important because each component has an interpretation that a non-specialist analyst can reason about. For example, an analyst can ask whether the forecast needs a changepoint, whether weekly seasonality is too strong, whether a holiday effect is missing, or whether some observations should be treated as outliers.

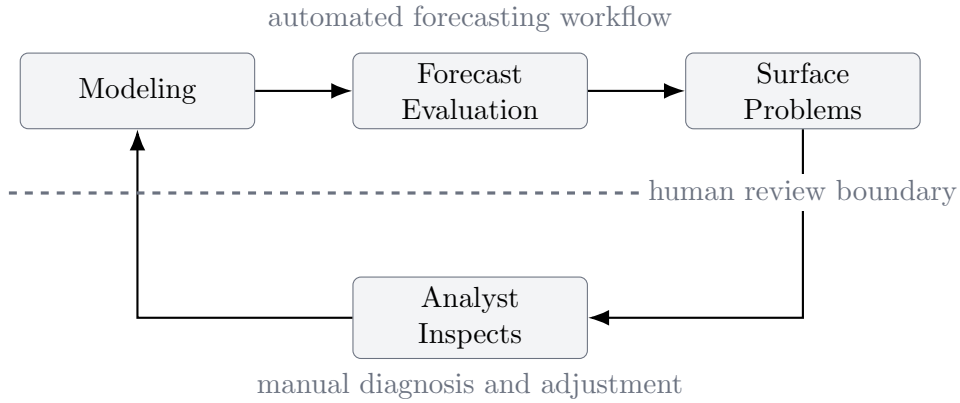


Figure 1: Analyst-in-the-loop forecasting. Automation handles model fitting, backtesting, and large-scale evaluation. The analyst is reserved for diagnosis and intervention.

1.3 Relevance in the Era of LLMs and Time-Series Foundation Models

The analyst-in-the-loop idea remains relevant today for two reasons. First, modern forecasting models have become more powerful, but real-world failure modes have not disappeared. Time-series foundation models such as Chronos and TimesFM show that pretraining can be useful for forecasting across many domains [2, 3]. However, even strong models can fail when the input data changes, contains unmarked interventions, or violates the assumptions learned during training.

Second, LLMs are well suited to the *analyst* part of the loop, not necessarily the numerical forecasting part. A generic LLM should not be trusted to extrapolate a numeric sequence without external tools and validation. Recent research has questioned whether language models are intrinsically useful as direct time-series forecasters [4]. The stronger design is to let statistical, machine-learning, or foundation forecasting models produce candidate forecasts, while the LLM agent orchestrates tools, reads diagnostics, explains likely causes, proposes bounded interventions, and validates them through backtesting.

Thus, the modern version of the Prophet workflow is:

Use models for forecasting, tools for diagnostics, and an LLM agent for structured analysis, intervention planning, and explanation.

1.4 Existing Forecast-Agent Work and Differentiation

The broad idea of AI agents for forecasting is already visible in current industry and open-source systems. Aera Technology describes a closed-loop forecasting process where humans and intelligent agents collaborate through automation, explanations, nudges, planner adjustments, bias evaluation, and continuous learning [5]. TimeCopilot describes itself as a GenAI forecasting agent that combines LLM reasoning with multiple time-series foundation models for model selection, validation, explanation, and natural-language forecasting [6]. Dynatrace also provides a Forecast Agent for creating forecasts and summarizing trends using natural-language input inside operational workflows [7].

This project is therefore best framed as a **focused implementation and evaluation** of an existing direction. The differentiating scope is narrower and more academic: the agent acts as a bounded junior analyst for Prophet-style forecast diagnosis and repair. It focuses on drift, changepoints, outliers, baseline comparison, intervention recommendation, and backtest-based validation. The main contribution is the controlled diagnostic loop and the evaluation of whether that loop helps.

2 Proposal: Agent-in-the-Loop Forecasting

2.1 Project Goal

The proposed course project is to build an **Agent-in-the-Loop Forecasting** system. The system will contain an LLM-based agent that acts as a junior or amateur forecasting analyst. The agent will not be given unrestricted authority and will not replace statistical evaluation. Instead, it will operate through a fixed set of tools and will be required to justify every intervention using measurable evidence from backtests and diagnostics.

2.2 Research Question

The core question is:

Can a bounded LLM agent improve a forecasting pipeline by diagnosing common time-series failure modes, recommending interventions, and validating those interventions through backtesting?

This can be decomposed into smaller questions:

- Can the agent correctly identify common failure modes such as drift, changepoints, and outliers?
- Can the agent recommend reasonable interventions without overfitting?
- Can the agent improve forecast accuracy or reliability compared with default automated models?
- Can the agent produce explanations that are useful to a human reviewer?

2.3 System Architecture

The proposed system has five major components:

1. **Data layer:** one or more time-series datasets with timestamps, target values, and optional metadata.

2. **Forecasting model layer:** baseline models such as Seasonal Naive, Prophet, AutoARIMA, ETS, and optionally a modern model such as Chronos or TimesFM if time permits.
3. **Diagnostics layer:** tools for drift detection, changepoint detection, outlier detection, and metric computation.
4. **LLM analyst agent:** an agent that reads diagnostics, chooses tool calls, proposes bounded interventions, and writes explanations.
5. **Evaluation layer:** backtesting that accepts or rejects proposed interventions based on forecast metrics.

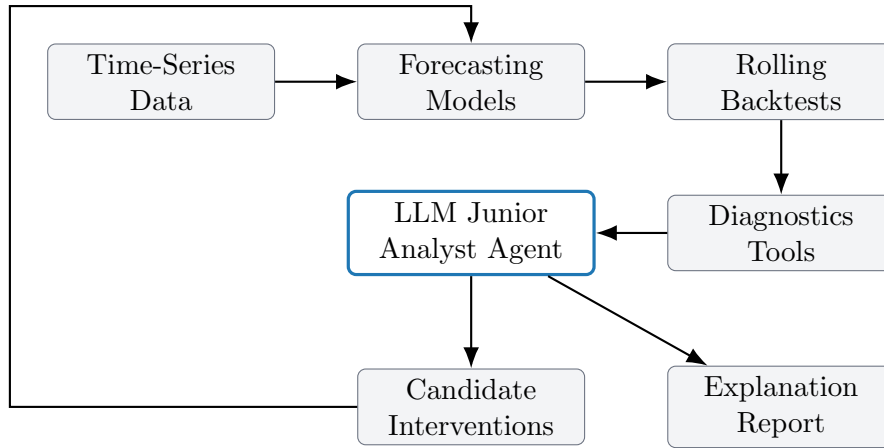


Figure 2: Agent-in-the-loop architecture. The LLM agent acts as a bounded analyst that uses diagnostic tools and backtesting evidence before recommending changes.

2.4 Agent Role and Constraints

The agent is intentionally framed as a junior analyst. It has limited capabilities and must use tools rather than relying on unsupported intuition. This is an important design choice because forecasting decisions must be validated numerically: an explanation may sound plausible but still produce a worse forecast.

The agent should be constrained as follows:

- It must inspect metrics and plots before recommending a change.
- It must describe the detected failure mode.
- It may only choose from a predefined intervention menu.
- It must rerun backtests after an intervention.
- It must compare before and after metrics.
- It must reject its own recommendation if backtest performance worsens.
- It must produce a final explanation that separates evidence from speculation.

2.5 Capabilities

The following capabilities are listed in implementation priority order. The project can remain useful even if only the first few are implemented well.

Table 1: Capability roadmap for the junior forecasting analyst agent.

Order	Capability	Purpose
1	Detect drift	Identify whether the data-generating process or residual distribution has changed.
2	Identify changepoints	Locate abrupt changes in trend level or slope.
3	Flag outliers	Detect isolated observations that may distort model fitting or evaluation.
4	Compare baseline models	Determine whether the current model is worse than simple alternatives.
5	Recommend interventions	Propose bounded fixes such as adding changepoints, outlier treatment, or model switching.
6	Validate through back-testing	Accept only changes that improve out-of-sample performance or reliability.
7	Produce explanation report	Summarize evidence, decision, intervention, and remaining uncertainty.

2.5.1 Capability 1: Drift Detection

Drift occurs when the statistical behavior of a time series changes over time. In forecasting, this may appear as a shift in the target distribution, a change in residual distribution, or a deterioration in forecast error after a particular date. Drift matters because a model trained on the past may no longer represent the current system.

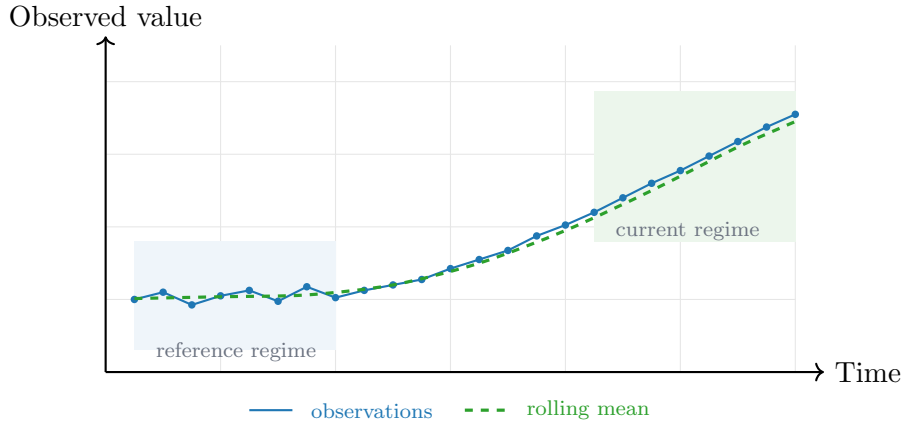


Figure 3: Illustration of drift. The early period is stable around one level, while the later period slowly moves to a different range without a single obvious break date. A model trained only on the early period may underforecast the later period.

2.5.2 Capability 2: Changepoint Identification

A changepoint is a time at which the structure of the series changes. This may be a change in level, slope, variance, or seasonality.

The agent’s job is not merely to find a date. It should interpret the changepoint in relation to forecast error. For example, if forecast error rises sharply after a detected changepoint, the agent may recommend increasing changepoint flexibility, explicitly adding a changepoint, or training separate models before and after the break.

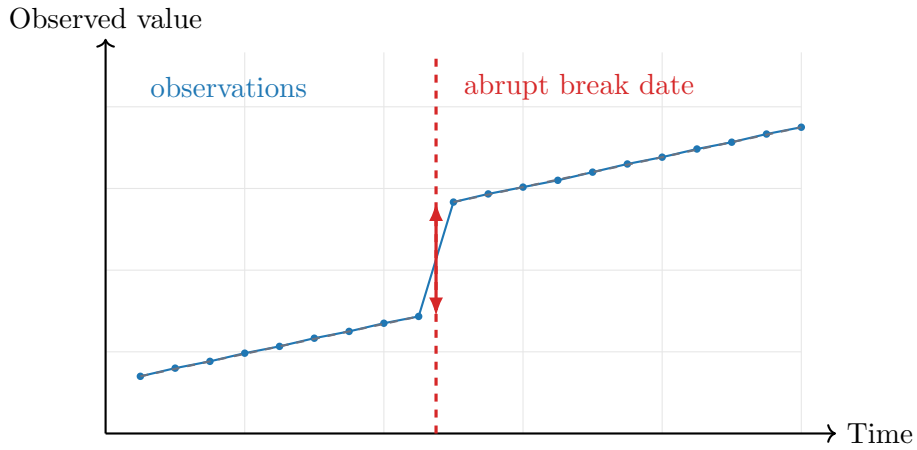


Figure 4: Illustration of a changepoint. The series has an abrupt level change at a specific time. Unlike gradual drift, the break can be localized to a narrow time window.

2.5.3 Capability 3: Outlier Flagging

Outliers are isolated observations that are inconsistent with the local behavior of the series. They may be caused by measurement errors, temporary shocks, logging bugs, one-off events, or real but non-recurring business events. Outliers can distort trend estimation, widen prediction intervals, and create misleading backtest errors.

The agent should distinguish between isolated outliers and structural changes. A single extreme point may justify outlier treatment. A persistent shift should be treated as drift or a changepoint instead.

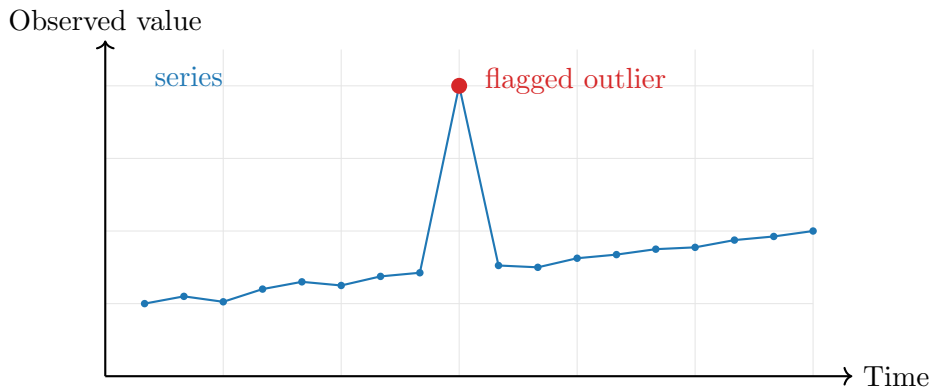


Figure 5: Illustration of an outlier. The red observation is far from the local pattern but the surrounding observations return to the prior trajectory.

2.5.4 Capability 4: Baseline Model Comparison

Forecasting systems should always compare against simple baselines. A sophisticated model that cannot beat Seasonal Naive is not useful. The project will compare some combination of the following models:

- Seasonal Naive as a simple reference model.
- Prophet as the interpretable analyst-in-the-loop baseline.
- AutoARIMA or ETS as classical statistical baselines.

- A deep-learning or foundation-model baseline if time permits.

Metrics may include MAE, RMSE, MASE, WAPE, sMAPE, and prediction interval coverage. The agent should use these metrics to decide whether a proposed intervention is actually helpful.

2.5.5 Capability 5: Intervention Recommendation

The agent should recommend interventions from a bounded menu. This keeps the system testable and reduces the risk of arbitrary LLM behavior. Candidate interventions include:

- Add or relax changepoints.
- Switch between additive and multiplicative seasonality.
- Add weekly, yearly, or custom seasonality.
- Treat a short interval as outlier-contaminated.
- Add known holidays or event regressors.
- Prefer a different baseline model when Prophet is consistently worse.
- Retrain using a more recent window when drift is detected.

Each recommendation must include:

- the observed symptom,
- the likely cause,
- the proposed intervention,
- the metric that will determine whether the intervention is accepted.

2.5.6 Capability 6: Validation Through Backtesting

Backtesting is the **guardrail** that prevents the agent from producing plausible but harmful recommendations. An intervention is accepted only if it improves the chosen metric or improves reliability without unacceptable accuracy loss.

2.5.7 Capability 7: Explanation Report

The final output should be a concise report written by the agent. A useful report should contain:

- dataset summary,
- selected forecast horizon,
- baseline model comparison,
- detected issues,
- recommended and tested interventions,
- before and after metrics,
- final model recommendation,
- limitations and open questions for a human analyst.

3 Implementation Plan

3.1 Minimum Viable Project

The minimum viable version will focus on a small but complete loop:

1. Load one or more benchmark time-series datasets or custom-generated datasets.
2. Fit Prophet and Seasonal Naive baselines.
3. Run historical backtests.
4. Detect outliers, changepoints, and drift.
5. Ask the LLM agent to write a diagnosis and choose from a small intervention menu.
6. Refit after intervention and compare metrics.
7. Generate a final explanation report.

3.2 Extended Version

If time permits, the project can add:

- AutoARIMA, ETS, or StatsForecast baselines.
- Chronos or TimesFM as a modern foundation-model baseline.
- A small user interface for visual inspection.
- A chat interface for interacting with the junior analyst agent.
- Synthetic benchmark cases with known ground-truth drift, changepoints, and outliers.
- A comparison between rule-based diagnostics and LLM-agent diagnostics.

4 Agent Evals

The evaluation should test whether the system behaves like a careful junior analyst. The goal is not only to measure whether the final forecast improves, but also whether the agent follows the diagnostic workflow, uses evidence correctly, respects its constraints, and avoids unsupported recommendations.

4.1 Eval Dataset

The eval set can contain a mix of benchmark time-series datasets and custom-generated cases. Custom-generated cases are useful because they can include known ground-truth events such as drift, changepoints, outliers, missing values, and regime changes. Each eval case should contain the time series, forecast horizon, baseline model outputs, diagnostic-tool outputs, expected issue labels, allowed interventions, and final forecast metrics.

4.2 Agent-Level Evals

- **Diagnostic correctness:** Does the agent correctly identify drift, changepoints, and outliers when they are present? This can be measured using precision, recall, and false-positive rate on synthetic cases with known injected problems.
- **Tool-use correctness:** Does the agent call the required diagnostic and backtesting tools before making a recommendation? The agent should not invent evidence, cite metrics that were not computed, or skip validation steps.
- **Intervention quality:** Does the agent choose reasonable interventions from the permitted intervention menu? Examples include outlier treatment, adding known events, adjusting changepoint flexibility, changing seasonality assumptions, or falling back to a simpler baseline.
- **Backtest discipline:** Does the agent accept an intervention only when backtesting supports it? This eval should count both successful improvements and harmful recommendations that were correctly rejected by the backtest guardrail.
- **Forecast outcome:** Does the accepted intervention improve or preserve forecast quality compared with Prophet default, Seasonal Naive, and other baselines? Metrics can include MAE, RMSE, MAPE or sMAPE, and prediction interval coverage if probabilistic forecasts are used.
- **Constraint adherence:** Does the agent stay within its junior-analyst role? It should not claim causal explanations without evidence, use tools outside the allowed set, or make automatic production changes without validation.
- **Explanation quality:** Does the final report clearly state the detected issue, the evidence, the recommended intervention, the backtest result, and the remaining uncertainty? This can be graded with a small rubric or by human review.
- **Stability:** Does the agent give consistent diagnoses and recommendations across repeated runs on the same eval case? This is important because the workflow should be reliable enough for a human analyst to inspect and trust.

4.3 Comparison Baselines

The agent should be compared against simple non-agent baselines:

- Prophet default with no diagnostic loop.
- Seasonal Naive or another simple statistical baseline.
- A rule-based diagnostic workflow that applies fixed interventions without LLM reasoning.
- An LLM-only diagnosis without tool outputs, to show why tool grounding and backtesting are necessary.

5 Expected Contributions

The contribution is an agentic workflow for forecast diagnosis and repair. The system demonstrates how an LLM agent can be useful in a forecasting pipeline without pretending to be a numerical forecaster. This is a realistic and defensible use of agentic AI because the agent:

- observes model outputs and diagnostics,

- reasons over detected failure modes,
- calls tools,
- proposes interventions,
- validates decisions using backtesting,
- communicates results to a human reviewer.

6 Conclusion

The analyst-in-the-loop workflow remains relevant because real-world forecasting still requires diagnosis, context, and selective intervention. LLMs make this workflow more interesting by allowing part of the analyst role to be automated. The resulting system combines classical forecasting, modern agentic tool use, model monitoring, and explainable decision-making in a constrained, testable pipeline.

References

- [1] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *PeerJ Preprints*, 5:e3190v2, 2017. <https://doi.org/10.7287/peerj.preprints.3190v2>.
- [2] Abdul Fatir Ansari et al. Chronos: Learning the language of time series. 2024. <https://www.amazon.science/publications/chronos-learning-the-language-of-time-series>.
- [3] Abhimanyu Das et al. A decoder-only foundation model for time-series forecasting. Google Research Blog, 2024. <https://research.google/blog/a-decoder-only-foundation-model-for-time-series-forecasting/>.
- [4] Ming Jin Tan et al. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems*, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/file/6ed5bf446f59e2c6646d23058c86424b-Paper-Conference.pdf.
- [5] Aera Technology. Agentic decision intelligence for forecasting: A continuous learning loop between humans and AI. 2025. <https://www.aeratechnology.com/blogs/improving-continuous-forecasting-with-ai-forecast-agents/>.
- [6] TimeCopilot. The GenAI forecasting agent: LLMs and time-series foundation models. 2025. <https://timecopilot.dev/>.
- [7] Dynatrace. Forecast Agent. 2026. <https://www.dynatrace.com/hub/detail/forecast-agent/>.